SECUREHUB / VULNTRACK

PLATAFORMA DE GESTIÓN DE VULNERABILIDADES Y RIESGOS

INFORME TECNICO DEVSECOPS

SECUREHUB / VULNTRACK

PLATAFORMA DE GESTIÓN DE VULNERABILIDADES Y RIESGOS

INFORME TÉCNICO DE IMPLEMENTACIÓN DEVSECOPS

Título del proyecto	**SecureHub / VulnTrack**
Subtitulo	**Plataforma de Gestión de Vulnerabilidades y Riesgos**
Documento	**Informe Técnico de Implementación DevSecOps**
Equipo de Ingeniería	**Muñoz Rey Hilary Michelle** *Analista de Documentación Técnica* **Ruiz Camelo Jennifer Dayan** *Especialista en Documentación de Software* **Molina Lancheros Daniel Steven** *Ingeniero DevSecOps* **Torres Pérez Eduar De Jesús** *Desarrollador de Software* **Alexander Diaz Molano** *Analista Técnico de Arquitectura*
Evaluador Técnico	Ospina Navas Jaider *Magíster en Ciberseguridad y Ciberdefensa*
Fecha de entrega	23 de octubre de 2026
Licenciamiento	MIT License (Código abierto) *Copyright © 2026 Equipo VulnTrack*
Versión del documento	**V1.0 — revisión de evidencias** *(10/10/2026)*

1. INTRODUCCIÓN

1.1 Justificación de la aplicación elegida

La gestión de vulnerabilidades es una disciplina crítica dentro de cualquier estrategia DevSecOps moderna. Las organizaciones necesitan no solo identificar fallos de seguridad en sus activos tecnológicos, sino también gestionarlos de forma trazable: asociarlos a riesgos, planificar su remediación, y dejar evidencia auditable de cada decisión tomada.

SecureHub responde a esta necesidad mediante una plataforma web multi-inquilino que centraliza el ciclo de vida completo de una vulnerabilidad. El sistema permite registrar activos tecnológicos (servidores, equipos, aplicaciones, APIs y bases de datos), documentar hallazgos con referencia CVE, evaluar el riesgo asociado según probabilidad e impacto, y programar planes de remediación con seguimiento de SLAs.

Adicionalmente, el sistema se apoya en la API pública del NIST (NVD) para enriquecer automáticamente cada hallazgo con datos oficiales de severidad y puntuación CVSS, evitando la carga manual de información que ya está estandarizada a nivel global.

La aplicación se diseñó desde su origen bajo principios de aislamiento de datos por inquilino, autenticación reinforced y trazabilidad inmutable de eventos, lo que permite alojar múltiples empresas en una misma instancia sin comprometer la confidencialidad entre ellas.

1.2 Objetivos del trabajo

1.2.1 Objetivo general

Desarrollar y asegurar una plataforma web multi-inquilino para la gestión del ciclo de vida de vulnerabilidades, integrando prácticas DevSecOps a lo largo de su arquitectura, su modelo de datos y su pipeline de integración continua.

1.2.2 Objetivos específicos

- Implementar un modelo de aislamiento de datos por empresa utilizando Row Level Security (RLS) nativo de PostgreSQL.
- Proveer autenticación con JWT y segundo factor de autenticación TOTP obligatorio como política de seguridad interna.
- Modelar los activos, hallazgos, riesgos, planes de remediación y evidencias con integridad referencial completa.

- Automatizar el vencimiento de SLAs de remediación mediante un worker en segundo plano.
- Servir las evidencias documentales mediante un servidor estático endurecido que impida listado de directorios y métodos no permitidos.
- Enriquecer los hallazgos con información oficial del NIST a través de un proxy controlado.
- Exponer una API REST documentada con OpenAPI/Swagger.

1.3 Alcance y limitaciones

El presente informe cubre la arquitectura del sistema SecureHub, su modelo de datos, sus controles de seguridad implementados a nivel de aplicación y base de datos, el modelado de amenazas bajo STRIDE, la implementación del pipeline DevSecOps y los resultados de seguridad obtenidos a partir de las herramientas del pipeline.

El alcance corresponde a la implementación y validación del pipeline en GitHub Actions, con despliegue temporal de staging para DAST; no constituye evidencia de despliegue productivo. Los aspectos comerciales y el escalamiento cloud productivo quedan fuera del alcance.

1.4 Licenciamiento del software

SecureHub / VulnTrack se distribuye bajo la **licencia MIT**, un modelo de licenciamiento de código abierto que permite el uso, modificación, distribución y comercialización del software, sujeto a la conservación de los avisos de derechos de autor y de licencia.

El repositorio oficial incorpora el archivo LICENSE, en el cual se establece el aviso de derechos de autor correspondiente al año 2026 y al Equipo VulnTrack. Asimismo, el archivo README.md identifica expresamente la licencia seleccionada y proporciona acceso a sus condiciones.

Esta decisión facilita la reutilización, adaptación y evolución del proyecto, al tiempo que establece un marco claro para su distribución.

2. ARQUITECTURA

2.1 Visión general de la arquitectura

SecureHub está construido sobre una arquitectura de separación por capas que desacopla la interfaz de usuario, la lógica de negocio, la persistencia y el almacenamiento de archivos. Los componentes observables en el proyecto son los siguientes:

- **Frontend:** Aplicación React 19 montada sobre Vite 5, estilizada con Tailwind CSS v4, con iconografía provista por lucide-react y gráficas mediante recharts. Se organiza en un punto de entrada (main.jsx), un contenedor de autenticación (App.jsx), un contenedor principal post-login (Dashboard.jsx) y un conjunto de vistas modulares bajo views/.

- **Backend API:** Servidor Node.js con Express 4, protegido por helmet (cabeceras defensivas) y cors (control de origen), con límite de payload de 10 KB para prevenir DoS. La lógica se organiza en rutas y controladores por dominio funcional: autenticación, activos, usuarios, hallazgos, riesgos, remediación, dashboard, auditoría y roles.
- **Persistencia:** Base de datos PostgreSQL con extensión pgcrypto para generación criptográfica de UUIDs. El aislamiento entre inquilinos se resuelve con políticas de Row Level Security a nivel motor, no a nivel aplicación.

- **Servidor de archivos estáticos:** Contenedor Nginx dedicado a servir las evidencias documentales, endurecido con server_tokens off, autoindex off y restricción estricta de métodos HTTP a GET y HEAD.

- **Trabajador en background:** Proceso cron basado en node-cron que se ejecuta periódicamente para detectar planes de remediación vencidos, marcarlos como VENCIDO y dejar trazabilidad en la bitácora de auditoría.

La comunicación entre frontend y backend se realiza exclusivamente por HTTP/JSON, con autenticación mediante JWT transportado en el header Authorization: Bearer <token>.

2.2 Diagramas UML

2.2.1 Diagrama de casos de uso

La identificación de los actores y sus interacciones con la plataforma permite establecer el alcance funcional de SecureHub / VulnTrack y representar las operaciones asociadas a cada perfil de usuario.

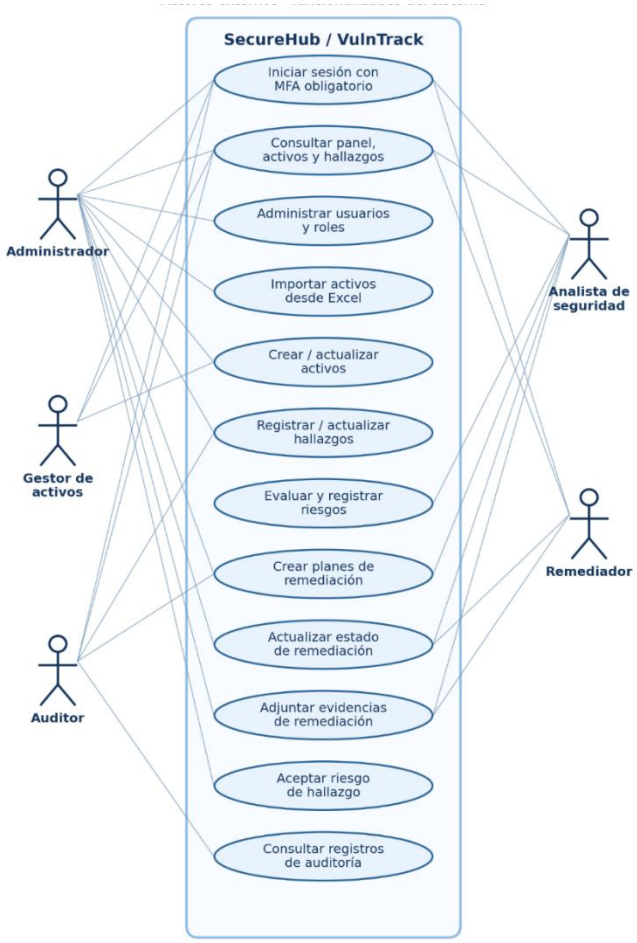


Figura 1. *Diagrama UML de casos de uso de SecureHub / VulnTrack.*

El diagrama de casos de uso representa las interacciones entre los actores del sistema y las funcionalidades asociadas a la administración de activos, la gestión de vulnerabilidades, el análisis de riesgos, la remediación y la consulta de información de auditoría. Su propósito es visualizar las responsabilidades funcionales de cada perfil y los mecanismos de acceso previstos en el diseño. Esta representación permite comprender el alcance operativo de la plataforma y constituye una referencia para la definición de requisitos funcionales y controles de autorización.

2.2.2 Diagrama de componentes

La arquitectura lógica de SecureHub / VulnTrack se organiza mediante componentes de software con responsabilidades específicas, cuya interacción permite desarrollar las funcionalidades de la plataforma.

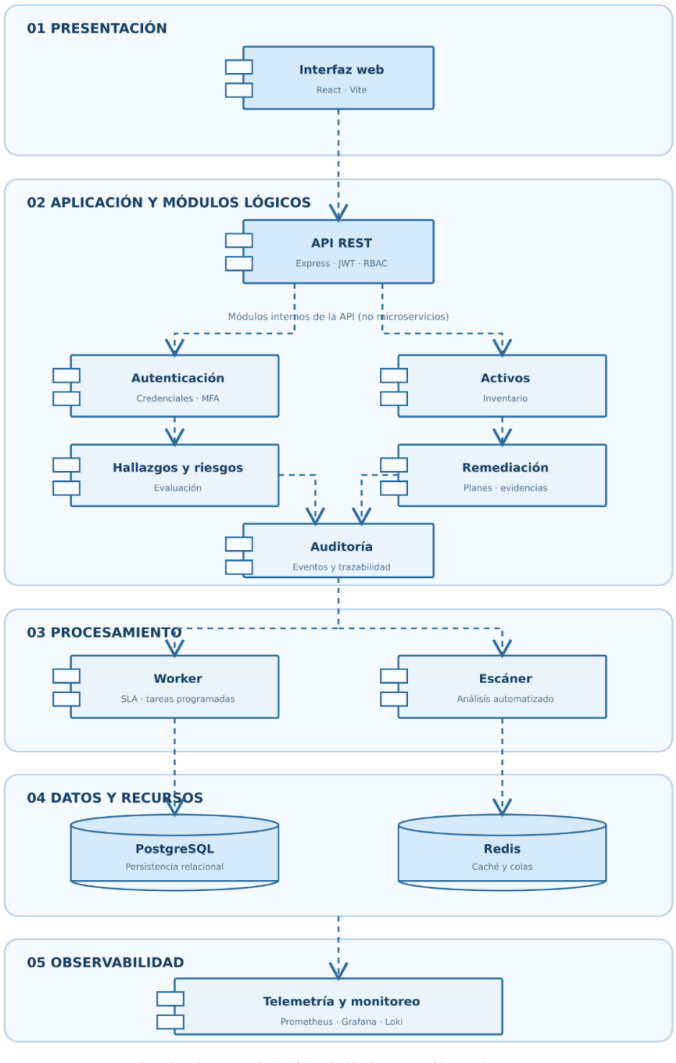


Figura 2. *Diagrama UML de componentes de SecureHub / VulnTrack.*

El diagrama de componentes describe la organización lógica de la solución, incluyendo la interfaz de usuario, los servicios de aplicación, los mecanismos de autenticación y los componentes de persistencia de información. Asimismo, permite identificar las dependencias entre los diferentes módulos y comprender la separación de responsabilidades dentro de la arquitectura. Esta representación facilita el análisis de la modularidad del sistema, su mantenibilidad y las posibilidades de evolución e integración con servicios adicionales.

2.2.3 Diagrama de Entidad Relación

El modelo de datos de SecureHub / VulnTrack se estructura mediante entidades relacionadas que permiten administrar la información correspondiente a usuarios, activos tecnológicos, vulnerabilidades, riesgos y procesos de remediación.

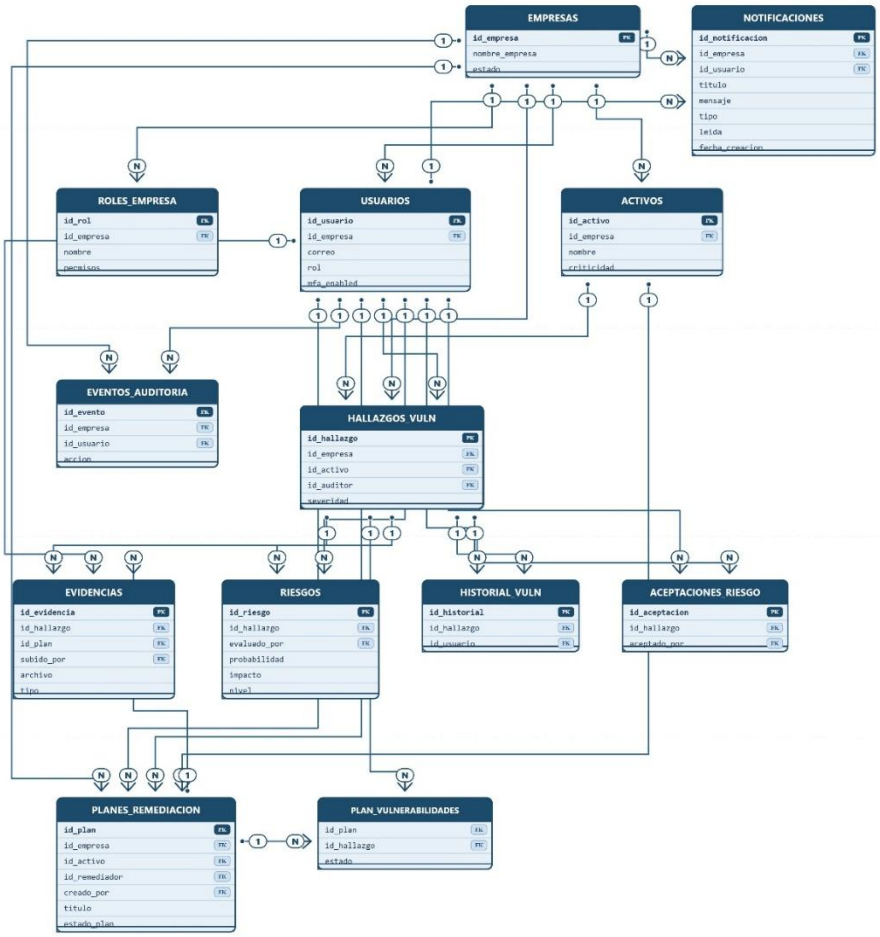


Figura 3. *Modelo entidad–relación de la base de datos de SecureHub / VulnTrack*

El diagrama entidad–relación representa la estructura lógica de la base de datos PostgreSQL utilizada por la plataforma, identificando las entidades, sus atributos principales y las relaciones de cardinalidad existentes. El modelo permite comprender cómo se organiza la información relacionada con la identificación de activos, el registro de vulnerabilidades, la evaluación de riesgos, el seguimiento de acciones de remediación y la trazabilidad de las operaciones. Esta representación constituye una referencia para analizar la integridad y consistencia de los datos, así como las dependencias entre los diferentes módulos del sistema.

2.2.4 Diagrama de secuencia

Como parte del diseño de los mecanismos de protección de acceso, se contempla un proceso de autenticación multifactor (MFA) orientado a fortalecer la verificación de identidad de los usuarios.

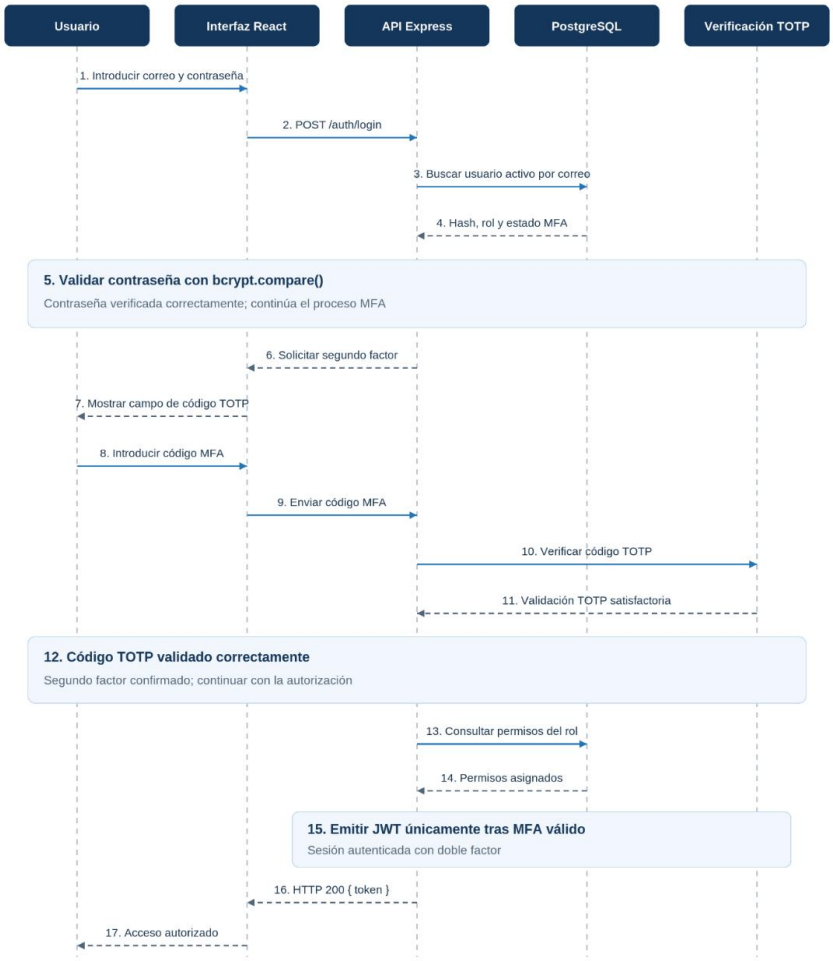


Figura 4. *Diagrama UML de secuencia - proceso de autenticación multifactor (MFA).*

El diagrama de secuencia representa el orden temporal de los mensajes intercambiados durante un proceso exitoso de autenticación multifactor. El flujo contempla la validación inicial de credenciales, la verificación de un segundo factor de autenticación y la autorización de acceso una vez completadas las comprobaciones previstas. Esta representación permite analizar las responsabilidades de los componentes involucrados y establecer una referencia para la implementación y validación de controles de identidad y acceso. El flujo corresponde al diseño propuesto y no constituye, por sí mismo, evidencia de su implementación operativa.

2.2.5 Diagrama de despliegue

Permite identificar la distribución de los componentes de software y sus relaciones con el entorno de ejecución.

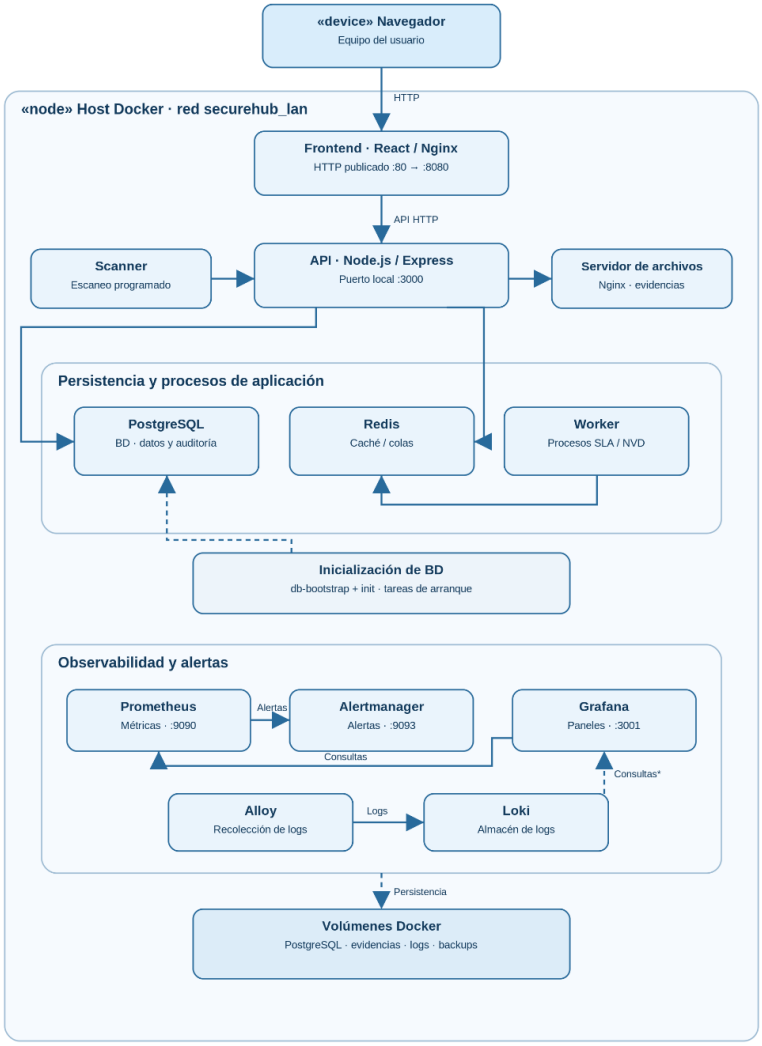


Figura 5. *Diagrama UML de despliegue de SecureHub / VulnTrack.*

El diagrama de despliegue representa la distribución de los componentes de la plataforma sobre su infraestructura de ejecución, incluyendo los servicios de aplicación, los contenedores y los recursos de persistencia de datos. Permite visualizar las relaciones de comunicación entre los diferentes elementos y comprender cómo se organiza el entorno tecnológico de la solución. Esta representación sirve como referencia para analizar aspectos de implementación, aislamiento de servicios, administración de dependencias y posibles estrategias de escalabilidad y disponibilidad.

2.3 Modelo de datos

El esquema relacional se compone de nueve tablas principales más una tabla auxiliar de roles personalizados por empresa. Todas las tablas de negocio incorporan una columna id_empresa que las vincula al inquilino propietario, requisito indispensable para que las políticas de Row Level Security puedan filtrar los registros por contexto.

Tablas del esquema:

- empresas: Registro de inquilinos. Campo estado booleano para soft delete.
- usuarios: Usuarios de cada empresa. correo único, password_hash, campos mfa_secret y mfa_enabled.
- activos: Inventario tecnológico. tipo libre, criticidad restringida por CHECK a BAJA/MEDIA/ALTA/CRITICA.
- hallazgos_vuln: Vulnerabilidades detectadas. cve_id, severidad con CHECK, cvss decimal, estado_hallazgo con CHECK, id_auditor FK a usuarios.
- hallazgos_activos: Relación N:M entre hallazgos y activos. PK compuesta, id_empresa explícito para RLS.
- riesgos: Evaluación de riesgo por hallazgo. probabilidad, impacto y riesgo_global derivado.
- planes_remediacion: Planes de acción correctiva. id_remediador FK, fecha_limite, estado_plan con CHECK.
- evidencias: Documentos adjuntos a hallazgos. ruta_archivo, subido_por FK.
- eventos_auditoria: Bitácora de trazabilidad. detalles en JSONB, ip_origen, pensada como registro append-only.
- roles_empresa: Roles personalizados por inquilino. permisos en JSONB, referenciada en el script seed_roles.sql.

Decisiones de diseño observables:

- **Soft delete generalizado:** todas las tablas de negocio usan una columna estado BOOLEAN en lugar de eliminación física, preservando la trazabilidad histórica.

- **Aislamiento por RLS:** las políticas comparan id_empresa::text contra la variable de sesión app.current_tenant, con un valor especial BYPASS reservado para operaciones de autenticación y del worker de fondo.

- **UUID criptográficos:** el uso de gen_random_uuid() de pgcrypto evita la previsibilidad de identificadores secuenciales.

- **Restricciones CHECK:** se aplican a severidades, criticidades, estados y probabilidades, garantizando que los valores almacenados pertenezcan a un dominio conocido.

- **Usuario de base de datos restringido:** la aplicación se conecta como app_user (no superusuario), lo cual es indispensable para que las políticas RLS se apliquen realmente.

2.4 Arquitectura del pipeline DevSecOps

2.4.1 Diagrama del pipeline

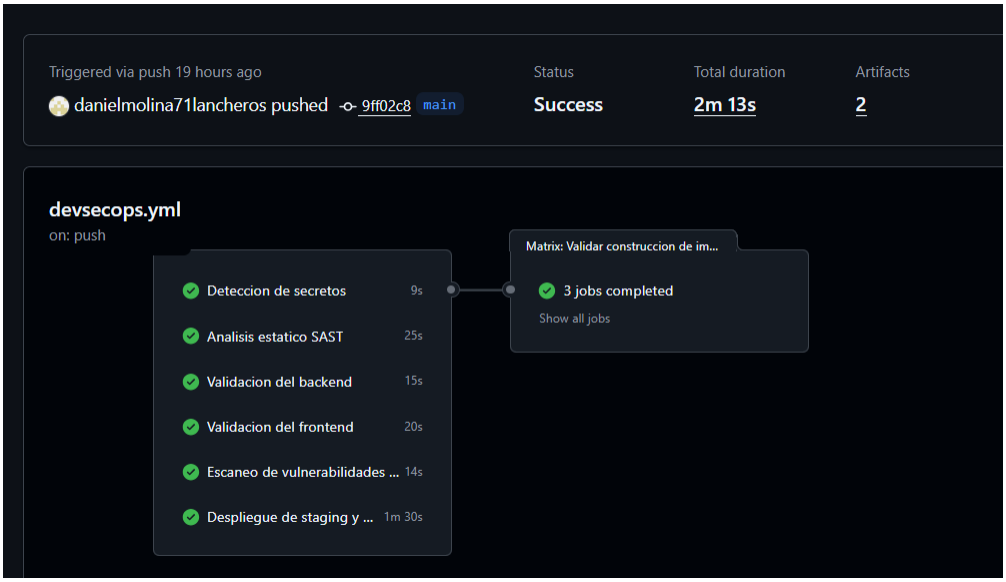


Figura 6. *GitHub Actions, ejecución #30: resumen del pipeline y trabajos completados.*

El grafo de GitHub Actions de la ejecución #30 muestra detección de secretos, SAST, validación backend, validación frontend, Trivy, despliegue temporal de staging con DAST y una matriz de tres validaciones de imágenes Docker.

2.4.2 Justificación de la arquitectura elegida

Los controles se ejecutan de manera automatizada y los trabajos de construcción dependen de comprobaciones previas. La separación de SAST, secretos, validación funcional, escaneo de dependencias y DAST mejora la trazabilidad del fallo y permite evidenciar cada fase en GitHub Actions.

3. MODELADO DE AMENAZAS

3.1 Metodología

El modelado de amenazas se realiza siguiendo la metodología STRIDE, con apoyo de la herramienta OWASP Threat Dragon para la construcción del diagrama de flujo de datos y la identificación sistemática de amenazas sobre cada componente del sistema.

3.2 Alcance del modelado

Los activos considerados dentro del alcance del modelado son los siguientes, todos ellos identificables directamente en la implementación:

- La API REST expuesta en el puerto 3000.
- El servidor de base de datos PostgreSQL y su esquema multi-inquilino.
- El contenedor Nginx que sirve las evidencias documentales.
- Los tokens JWT emitidos por el sistema.
- Los secretos TOTP almacenados para MFA.
- Los registros de auditoría almacenados en eventos_auditoria.

3.3 Resultados de OWASP Threat Dragon

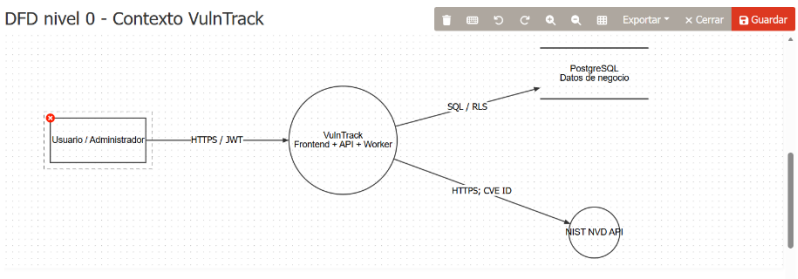


Figura 7. *Modelo de amenazas en OWASP Threat Dragon.*

Se validó en OWASP Threat Dragon 2.6.2 un único archivo JSON consolidado con dos diagramas: DFD nivel 0 (contexto; 7 celdas y 3 amenazas) y DFD nivel 1 (servicios; 17 celdas y 5 amenazas). Total: 8 amenazas registradas en el modelo. Sus estados se encuentran marcados como “Mitigated”; esta etiqueta expresa la valoración registrada en el modelo, no una verificación independiente de eficacia de los controles.

3.4 Amenazas identificadas por categoría STRIDE

- **Spoofing (Suplantación mediante robo de credenciales / Endpoint /api/auth/login):** Segundo factor TOTP obligatorio mediante speakeasy, con mfa_enabled activable y verificable.

- **Spoofing (Falsificación de tokens de sesión / Middleware de autenticación):** Verificación de firma JWT con process.env.JWT_SECRET y expiración de 8 horas.

- **Tampering (Modificación no autorizada de registros de auditoría / Tabla eventos_auditoria):** Diseñada como bitácora append-only; la aplicación no expone endpoints de modificación ni borrado.

- **Tampering (Manipulación de payloads hacia la API / Todas las rutas POST/PUT):** Validaciones de esquema en controladores + límite de payload de 10 KB.

- **Repudiation (Negación de acciones realizadas por un usuario / Operaciones críticas del sistema):** Helper registrarAuditoria invocado en creación de activos, hallazgos, riesgos, planes, usuarios y roles.

- **Information Disclosure (Fuga de datos entre inquilinos / Todas las tablas de negocio):** Políticas RLS que filtran por app.current_tenant, aplicadas sobre un usuario de BD no superusuario.

- **Information Disclosure (Exposición de evidencias por listado de directorio / Contenedor Nginx):** autoindex off + try_files $uri $uri/ =404.

- **Information Disclosure (Exposición de información del servidor / Cabeceras HTTP):** helmet en el backend + server_tokens off en Nginx.
- **Denial of Service (Payloads gigantes que agoten memoria / API Express):** express.json({ limit: '10kb' }).

- **Denial of Service (Métodos HTTP peligrosos sobre archivos estáticos / Contenedor Nginx):** Filtro que retorna 405 ante cualquier método distinto a GET o HEAD.
- **Elevation of Privilege (Escalado de privilegios entre roles / Todas las rutas sensibles):** Middleware autorizarRoles que valida rol o permiso granular por endpoint.

- **Elevation of Privilege (Modificación de roles del sistema / Endpoints de roles):** Bloqueo explícito de modificación del rol ADMINISTRADOR.

3.5 Contramedidas implementadas

Las contramedidas se materializan en capas complementarias:

- **Capa de red y transporte:** cabeceras defensivas (helmet), CORS controlado, restricción de métodos en el servidor de archivos.

- **Capa de autenticación:** hash de contraseñas con bcryptjs y salt de 10 rondas, MFA TOTP opcional obligable, expiración de tokens en 8 horas.

- **Capa de autorización:** verificación de permisos granulares por endpoint, con soporte para roles personalizados por inquilino almacenados en roles_empresa.

- **Capa de datos:** Row Level Security a nivel motor, con contexto inyectado por sesión mediante set_config('app.current_tenant', $1, true).

- **Capa de trazabilidad:** bitácora de auditoría enriquecida con JSONB para detalles variables, dirección IP de origen y marca temporal automática.

4. IMPLEMENTACIÓN DEL PIPELINE

4.1 Descripción general del pipeline

El proyecto cuenta con tres workflows de GitHub Actions para automatizar pruebas y controles de seguridad. El workflow principal, devsecops.yml, se ejecuta ante cambios en la rama main, pull requests dirigidos a esa rama, etiquetas de versión con el formato v* y ejecuciones manuales. Incluye detección de secretos con Gitleaks, análisis estático con Semgrep, pruebas y validaciones del backend y frontend, auditoría de dependencias, escaneo de vulnerabilidades con Trivy y análisis dinámico con OWASP

ZAP sobre un entorno de staging. También valida la construcción de imágenes Docker y, al publicar una etiqueta de versión, publica las imágenes correspondientes.

El workflow infrastructure-security.yml valida los playbooks de Ansible, comprueba el modelo STRIDE de Threat Dragon y analiza archivos de infraestructura como código con Checkov. El workflow tests.yml ejecuta las pruebas automatizadas del backend con Jest. Estos workflows separan las comprobaciones de aplicación, infraestructura y pruebas para que puedan ejecutarse de forma automatizada durante el desarrollo.

4.2 Fase por fase

4.2.1 SAST (Bandit / Semgrep)

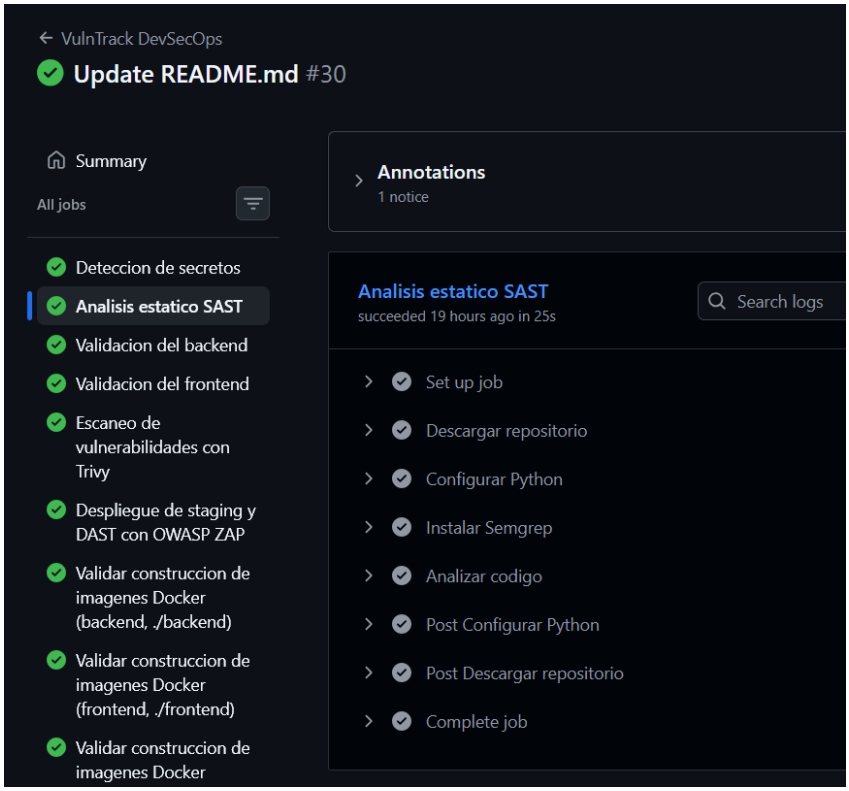


Figura 8. *Trabajo de análisis SAST: pasos de Semgrep completados.*

El análisis estático de seguridad se realiza con Semgrep sobre el código del backend y el frontend. El workflow instala la herramienta y ejecuta un análisis con la configuración automática de reglas, limitado a las carpetas backend y frontend. La opción --error hace que el job falle cuando el análisis detecta hallazgos.

4.2.2 SCA (Trivy)

La revisión de dependencias se integra en el workflow principal mediante npm audit y Trivy. Para el backend, npm audit revisa las dependencias de producción y utiliza el nivel high como umbral de fallo. Para el frontend, audita las dependencias del proyecto con el mismo umbral. Además, Trivy analiza el repositorio en busca de vulnerabilidades de severidad alta o crítica; en la ejecución revisada se registró una advertencia de parámetro trivy-args no reconocido, por lo que no se acredita la exclusión prevista de directorios.

4.2.3 Secret Scanning (Gitleaks)

La detección de secretos se realiza con Gitleaks en el workflow devsecops.yml. Antes del análisis, el job descarga el historial completo del repositorio y valida la configuración de hooks de pre-commit. Gitleaks busca posibles secretos expuestos en el código y en el historial disponible

4.2.4 Container Scanning (Trivy)

El workflow contempla construir y escanear con Trivy tres imágenes Docker: backend, frontend y scanner. El escaneo busca vulnerabilidades de severidad alta o crítica y está configurado para hacer fallar el job si las encuentra. La publicación de imágenes versionadas y latest se realiza únicamente cuando el workflow se activa con una etiqueta cuyo nombre comienza con v.

4.2.5 DAST (OWASP ZAP)

El análisis dinámico se ejecuta con OWASP ZAP contra un entorno temporal de staging desplegado con Ansible y Docker Compose. Antes del análisis, el workflow comprueba la disponibilidad de la API y valida que la especificación OpenAPI contenga rutas. ZAP utiliza esa especificación para analizar la API y genera informes en formato HTML y JSON. El job falla si el informe contiene alertas de riesgo alto o crítico, y publica los informes como artefactos de la ejecución. Al finalizar, intenta eliminar el entorno temporal.

4.2.6 Otras fases

Como controles complementarios, el workflow principal ejecuta pruebas automatizadas del backend con Jest, reglas de ESLint sobre el backend y el frontend, y la compilación del frontend. El workflow de seguridad de infraestructura valida la sintaxis de los playbooks de Ansible, comprueba que el modelo de Threat Dragon incluya diagramas DFD de nivel 0 y nivel 1 con amenazas de las seis categorías STRIDE, y analiza archivos de infraestructura como código con Checkov. Existe además un workflow separado para las pruebas del backend con Jest. Estos controles están

configurados en el repositorio; los resultados de sus ejecuciones deben consultarse en GitHub Actions para cada ejecución correspondiente.

4.3 Fragmentos clave del workflow

Análisis SAST con Semgrep

```
- name: Analizar codigo
  run: semgrep scan --config auto --error --exclude 'backend/zap-reports/**' backend frontend
```

Este paso analiza el código del backend y del frontend. La opción --error hace que el job falle si Semgrep encuentra hallazgos que deben bloquear la ejecución.

Escaneo del repositorio con Trivy

```
- name: Escanear dependencias y configuraciones
  uses: aquasecurity/trivy-action@v0.36.0
  with:
    scan-type: fs
    scan-ref: .
    scanners: vuln
    severity: HIGH,CRITICAL
    exit-code: "1"
```

Este paso revisa el repositorio en busca de vulnerabilidades altas o críticas y devuelve un error si las encuentra.

Dependencias para construir y escanear imágenes

```
needs: [secret-scan, sast, backend, frontend, trivy, dast]
```

La construcción y el escaneo de imágenes Docker esperan a que terminen los controles indicados en needs. Si uno falla, el job dependiente no se ejecuta normalmente.

4.4 Integración con el repositorio y ramas

Los workflows están integrados con GitHub Actions y se activan según eventos del repositorio. devsecops.yml se ejecuta cuando hay cambios en main, pull requests dirigidos a main, etiquetas de versión con el formato v* o una ejecución manual. infrastructure-security.yml se ejecuta manualmente, en pull requests hacia main y cuando se modifican en main archivos de infraestructura definidos en sus filtros de ruta. tests.yml

se ejecuta ante cambios en main o master, en pull requests dirigidos a esas ramas y mediante ejecución manual.

5. RESULTADOS DE SEGURIDAD

5.1 Resumen ejecutivo de hallazgos

Base de evidencia: ejecución #30 de GitHub Actions, commit 9ff02c8 (main), estado Success, duración 2 min 13 s y dos artefactos publicados. Los resultados revisados corresponden a la ejecución y los artefactos compartidos, no a una evaluación exhaustiva de toda la superficie de ataque.

Clasificación de resultados: 0 secretos identificados por Gitleaks; 0 hallazgos SAST Semgrep; 0 vulnerabilidades HIGH/CRITICAL reportadas por Trivy en los objetivos analizados; 1 alerta informativa ZAP. Se registra observación de configuración para Trivy.

El pipeline integra controles automáticos para detectar secretos, analizar el código y las dependencias, ejecutar pruebas y revisar la API y las imágenes de contenedor. La configuración establece condiciones para bloquear la ejecución ante determinados hallazgos, como resultados de severidad alta o crítica en algunos escaneos. Los resultados detallados —incluidos el número, tipo y estado de los hallazgos— deben consolidarse a partir de los informes generados por las herramientas. En este documento se describen los controles configurados y se identifican como pendientes los resultados que aún no cuentan con evidencia detallada.

5.2 Reportes por herramienta

5.2.1 Gitleaks

Resultado revisado: el artefacto gitleaks-results.sarif no registra hallazgos. Este resultado no descarta secretos fuera del historial y los archivos efectivamente examinados.

Gitleaks está configurado para buscar secretos expuestos en el repositorio. Antes del escaneo, el workflow descarga el historial completo y valida la configuración de pre-commit. El job se utiliza como control preventivo para detectar credenciales u otros valores sensibles que pudieran haberse incorporado al código. El detalle de hallazgos se documenta a partir del reporte de ejecución correspondiente.

5.2.2 Trivy (SCA + contenedores)

Resultado revisado del escaneo de filesystem: dos objetivos package-lock.json (backend y frontend), 0 vulnerabilidades de severidad HIGH/CRITICAL en el filtro ejecutado. El log mostró “Unexpected input(s) trivy-args”; no se confirma que las exclusiones configuradas por ese parámetro surtieran efecto. El estado exitoso de los trabajos de construcción de imágenes no sustituye los reportes detallados de escaneo por imagen.

El pipeline complementa la auditoría de dependencias de npm con escaneos de Trivy. npm audit revisa las dependencias del backend y del frontend y utiliza el nivel alto (high) como umbral para marcar un error. Trivy analiza el repositorio en busca de vulnerabilidades, con severidad alta o crítica, y omite directorios de dependencias instaladas y archivos generados.

Además, el job docker-build construye las imágenes del backend, frontend y scanner, y escanea cada imagen con Trivy para identificar vulnerabilidades altas o críticas. Este job depende de la finalización satisfactoria de varios controles previos. Los resultados concretos de cada análisis deben tomarse de los registros de GitHub Actions.

5.2.3 OWASP ZAP

Resultado revisado: 1 alerta de nivel informativo “Storable and Cacheable Content” en GET http://127.0.0.1:13000/api-docs.json (CWE-524, confianza media). No corresponde clasificarla como vulnerabilidad alta sin análisis adicional.

OWASP ZAP realiza un análisis dinámico de la API desplegada en un entorno temporal de staging. El workflow valida primero que la API responda y que su especificación OpenAPI incluya rutas. Luego, ZAP usa esa especificación para examinar los endpoints y genera informes en formato HTML y JSON, que se guardan como artefactos de la ejecución. El pipeline está configurado para bloquear el job si el análisis reporta alertas de riesgo alto o crítico. Los tipos y cantidades de alertas se deben obtener de los informes generados.

5.2.4 Bandit / Semgrep

Resultado comprobado: Semgrep completó el análisis de 80 archivos rastreados por Git y 234 reglas con 0 hallazgos y 0 bloqueantes; seis archivos fueron excluidos por .semgrepignore. Bandit no aparece en los trabajos ni pasos de la ejecución #30; no se atribuyen resultados a esa herramienta.

Semgrep analiza estáticamente el código del backend y del frontend mediante reglas seleccionadas con su configuración automática. El workflow excluye los reportes de ZAP y utiliza la opción --error, por lo que los hallazgos que Semgrep clasifica como bloqueantes hacen fallar el job. Los resultados detallados —regla activada, archivo, línea y severidad— deben documentarse a partir del registro de análisis de GitHub Actions.

5.3 Tabla consolidada de hallazgos

Gitleaks: 0 hallazgos en el artefacto SARIF revisado. Semgrep: 0 hallazgos, 234 reglas ejecutadas sobre 80 archivos rastreados por Git; 6 archivos ignorados por .semgrepignore. Trivy (filesystem): 0 vulnerabilidades HIGH/CRITICAL en los objetivos reportados backend/package-lock.json y frontend/package-lock.json; advertencia por entrada trivy-args no reconocida. OWASP ZAP: 1 alerta informativa de contenido almacenable y cacheable en GET /api-docs.json (CWE-524), sin alerta alta o crítica en el artefacto examinado. Bandit: sin evidencia de ejecución en el workflow #30. Los hallazgos nulos solo se refieren al alcance y configuración de cada herramienta.

5.4 Lecciones aprendidas y acciones de mejora

La evidencia debe diferenciar configuración, ejecución y resultado. Se recomienda: corregir el parámetro trivy-args y verificar las exclusiones realmente aplicadas; mantener un inventario de archivos ignorados por Semgrep; revisar cabeceras Cache-Control de /api-docs.json según sensibilidad de la información; agregar pruebas de aislamiento entre empresas y permisos RBAC/RLS; conservar artefactos SARIF/HTML/JSON junto con el identificador de commit y la fecha de ejecución.

6. MONITOREO Y OBSERVABILIDAD

Esta sección documenta las capacidades de supervisión técnica evidenciadas mediante capturas del entorno local de SecureHub / VulnTrack. Las imágenes muestran consultas de métricas, exploración de registros y evaluación de reglas de alerta. La evidencia corresponde a una sesión de observación y no acredita por sí sola operación continua en producción

6.1 Stack de observabilidad

El entorno observado integra Prometheus como origen de métricas y motor de evaluación de reglas; Grafana como interfaz de visualización y análisis; y Loki como fuente de datos para la consulta de registros. Las capturas de Grafana muestran consultas PromQL dirigidas a Prometheus y una vista de exploración de logs asociada a Loki. No se dispone en estas imágenes de una vista completa de la topología de recolección ni de pruebas de envío de notificaciones

6.2 Dashboards implementados

Se identificó un dashboard denominado «Vulntrack» en Grafana, con paneles de disponibilidad, tráfico HTTP, latencia y consumo de memoria de la API. Las imágenes fueron tomadas en la vista de edición de cada panel, por lo que acreditan la configuración y visualización de sus consultas, pero no necesariamente la publicación definitiva del dashboard.

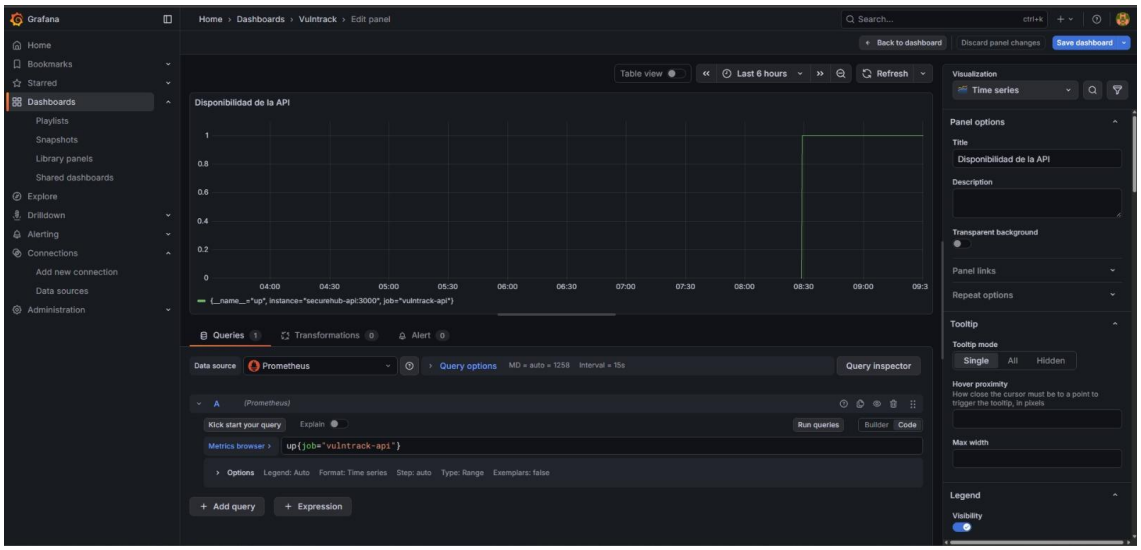


Figura 9. *Panel de disponibilidad de la API en Grafana.*

El panel consulta la expresión up{job="vulntrack-api"}. En el tramo visible, el valor mostrado es 1, lo que indica que Prometheus pudo recopilar métricas del objetivo durante ese intervalo. Esta observación no equivale a una medición de disponibilidad histórica ni garantiza que todos los endpoints de la API respondan correctamente.

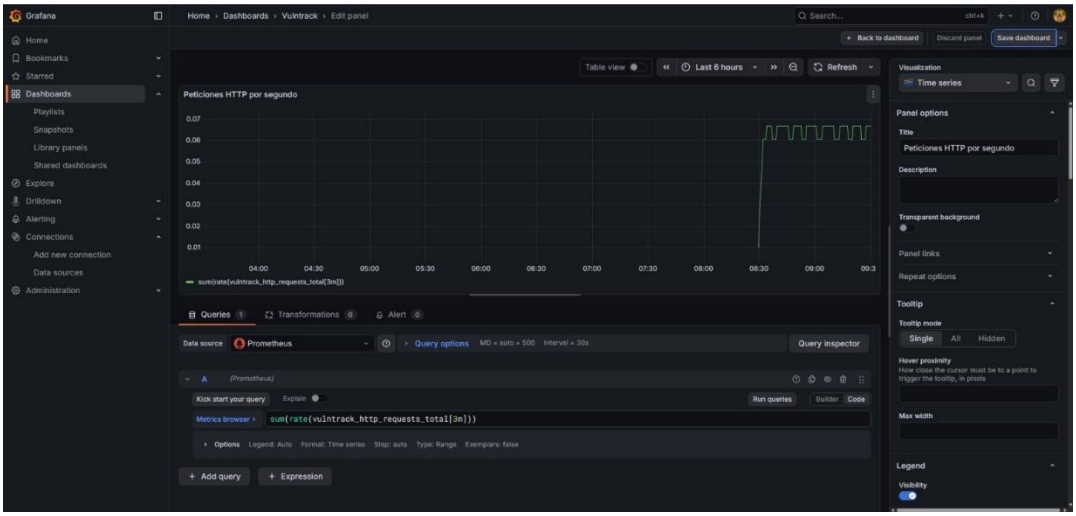


Figura 10. *Panel de peticiones HTTP por segundo.*

El panel utiliza sum(rate(vulntrack_http_requests_total[3m])) para estimar la tasa agregada de solicitudes HTTP a partir de una ventana móvil de tres minutos. La serie visible se sitúa aproximadamente entre 0,06 y 0,07 solicitudes por segundo en el período con datos, lo que permite observar la actividad de la API durante la captura.

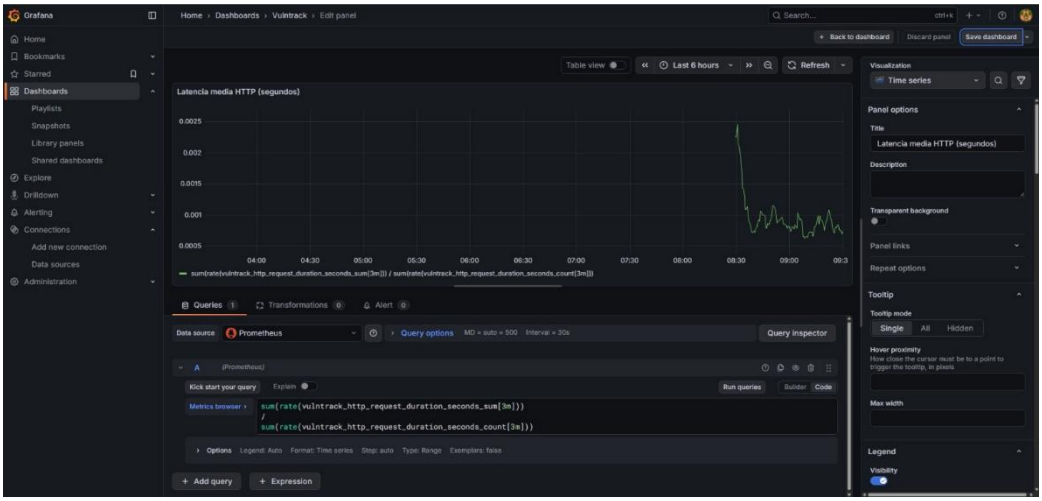


Figura 11. *Panel de latencia media HTTP de la API.*

La consulta divide la tasa de incremento acumulado de duración de solicitudes entre la tasa de solicitudes, empleando las métricas vulntrack_http_request_duration_seconds_sum y _count en una ventana de tres minutos. En el período visible, la latencia media estimada desciende desde un pico cercano a 0,0024 segundos hasta valores inferiores a 0,0012 segundos. La captura no permite extrapolar estos resultados a condiciones de carga elevada.

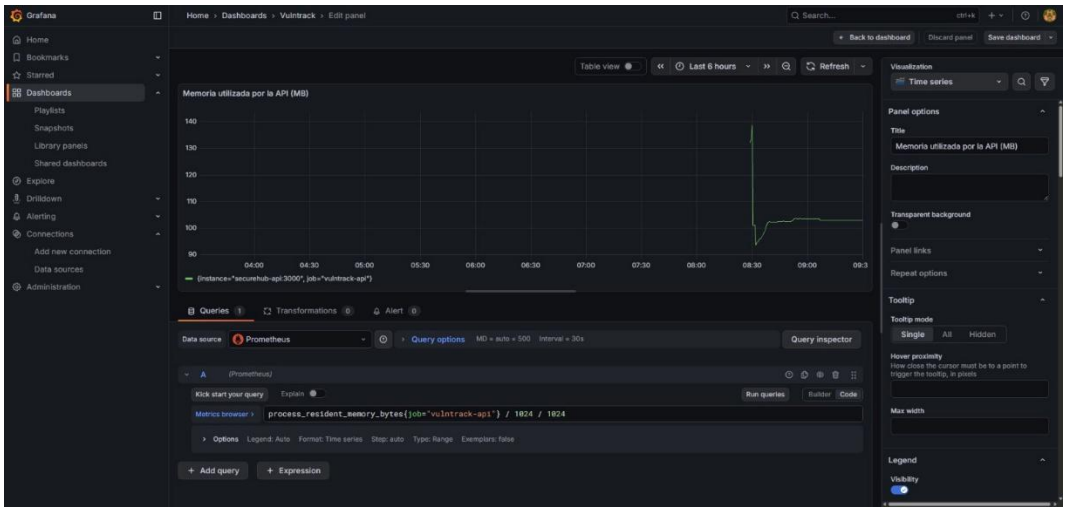


Figura 12. *Panel de memoria residente utilizada por la API.*

El panel presenta la métrica process_resident_memory_bytes{job="vulntrack-api"} convertida a mebibytes mediante división por 1024 × 1024, aunque el título del panel utiliza «MB». Tras una variación inicial, el consumo observado se estabiliza alrededor de 102–104 MiB. La gráfica permite seguir el comportamiento de memoria del proceso durante el intervalo representado.

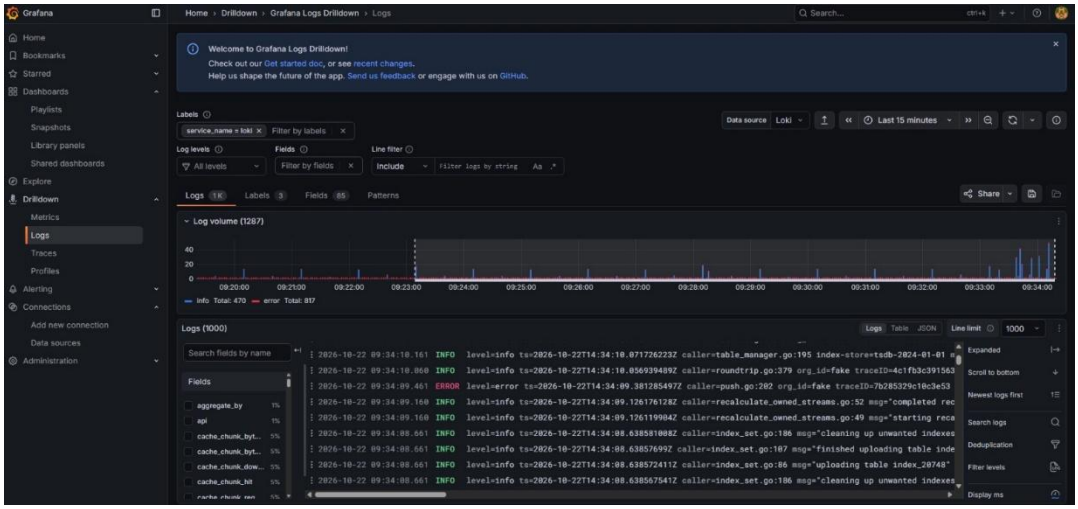


Figura 13. *Exploración de registros en Grafana con Loki.*

La vista de Grafana Logs Drilldown utiliza Loki como fuente de datos y muestra un volumen de 1.287 entradas en la ventana visible, con 470 registros de nivel info y 817 de nivel error según la clasificación de la interfaz. El filtro mostrado corresponde a service_name=loki, por lo que estos registros parecen pertenecer al propio servicio Loki; la captura no demuestra todavía la centralización de logs de la API VulnTrack. Es una evidencia válida de consulta de registros, con ese alcance específico.

6.3 Métricas y alertas relevantes

Prometheus presenta el grupo de reglas «vulntrack-api-alerts», cargado desde /etc/prometheus/alerts.yml, con cuatro reglas orientadas a supervisar disponibilidad, latencia, consumo de memoria y respuestas HTTP 5xx. Las capturas muestran las reglas evaluadas sin errores y en estado inactivo al momento de la consulta.

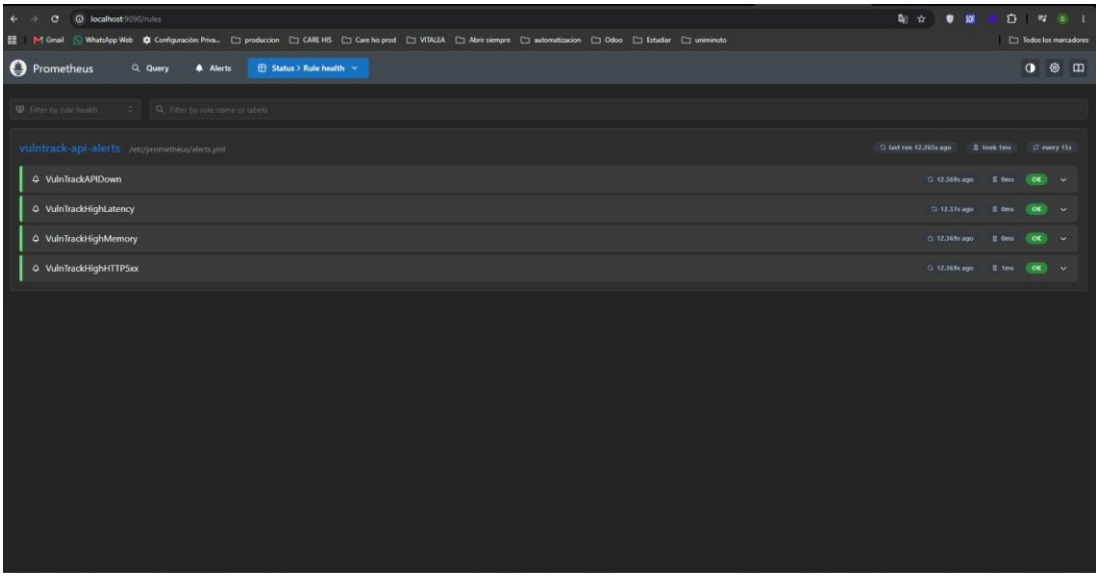


Figura 14. *Estado de evaluación de las reglas de alerta en Prometheus.*

En la vista «Rule health» aparecen cuatro reglas con estado OK: VulnTrackAPIDown, VulnTrackHighLatency, VulnTrackHighMemory y VulnTrackHighHTTP5xx. El indicador OK confirma que Prometheus evalúa las reglas sin errores en ese momento; no significa que se haya provocado una condición de alerta ni que se haya verificado su notificación.

Alcance de la evidencia

Las siete capturas respaldan la existencia de consultas de métricas, paneles de Grafana, acceso a registros mediante Loki y reglas de alerta configuradas en Prometheus. No documentan pruebas de carga, simulaciones de caída de servicio, activación de alertas, entrega de notificaciones ni monitoreo de largo plazo. Se recomienda conservar las capturas y, cuando sea posible, anexar el archivo de reglas y la configuración de fuentes de datos para facilitar la reproducibilidad.

7. CONCLUSIONES

7.1 Cumplimiento de objetivos

Se documentó la arquitectura multiempresa, el modelado STRIDE y la automatización de seguridad en GitHub Actions. La ejecución #30 del workflow devsecops.yml finalizó con estado Success, duración 2 min 13 s, commit 9ff02c8 en main y dos artefactos. Quedan por acreditar con evidencia específica los diagramas UML formales, dashboards de observabilidad y pruebas funcionales detalladas.

7.2 Aportes del proyecto

La solución integra un proceso trazable de gestión de vulnerabilidades y prácticas de seguridad en el ciclo de desarrollo. Los artefactos de Gitleaks y ZAP, los logs de Semgrep/Trivy y el modelo STRIDE permiten fundamentar el análisis en resultados reproducibles, diferenciando hallazgos detectados de controles propuestos.

7.3 Trabajo futuro

Como parte de su evolución tecnológica, SecureHub / VulnTrack contempla la incorporación progresiva de nuevas capacidades orientadas a fortalecer la gestión de vulnerabilidades, optimizar los procesos de remediación y ampliar las posibilidades de integración con infraestructuras empresariales.

Entre las principales líneas de evolución del producto se consideran:

- ✓ **Priorización inteligente de vulnerabilidades:** incorporar modelos de análisis que permitan establecer prioridades de remediación a partir de la criticidad de los activos, la exposición al riesgo y el contexto operativo de cada organización.

- ✓ **Integración con herramientas de seguridad:** ampliar la interoperabilidad mediante API con escáneres de vulnerabilidades, plataformas SIEM, sistemas de gestión de incidentes y herramientas de seguimiento de tareas.

- ✓ **Automatización de la remediación:** desarrollar flujos de trabajo que permitan asignar responsables, generar alertas, escalar incumplimientos y verificar el cierre de vulnerabilidades.

- ✓ **Analítica avanzada y tableros ejecutivos:** incorporar indicadores históricos, tendencias de riesgo, métricas de cumplimiento y reportes orientados a la toma de decisiones estratégicas.

- ✓ **Fortalecimiento de capacidades multiempresa:** evolucionar hacia mecanismos avanzados de administración por organización, políticas de seguridad configurables y segregación de información.

- ✓ **Escalabilidad y despliegue empresarial:** evaluar la incorporación de tecnologías de orquestación de contenedores, alta disponibilidad y mecanismos de recuperación ante fallos.

- ✓ **Inteligencia artificial aplicada a la ciberseguridad:** explorar asistentes inteligentes para interpretar hallazgos, sugerir estrategias de mitigación y apoyar el análisis técnico de vulnerabilidades.

Estas iniciativas constituyen una hoja de ruta para futuras versiones de SecureHub / VulnTrack y no representan funcionalidades implementadas en la versión evaluada. Su incorporación deberá acompañarse de procesos de validación técnica, pruebas de seguridad y evaluación de impacto operativo.

De esta manera, el proyecto establece una base para evolucionar desde una plataforma de gestión de vulnerabilidades hacia una solución integral de administración y análisis del riesgo de ciberseguridad

8. REFERENCIAS

OWASP Foundation. OWASP Threat Dragon (modelo STRIDE); OWASP ZAP (análisis dinámico). GitHub.

GitHub Actions, ejecución VulnTrack DevSecOps #30, commit 9ff02c8, 20/10/2026. Semgrep.

Documentación de análisis estático. Aqua Security. Trivy. Gitleaks. Documentación y formato SARIF.

Documentación interna SecureHub / VulnTrack: Manual 4.3 de Desarrollo (20/10/2026), Manual 4.4 de Despliegue y Operación y Manual 4.5 de Seguridad.